Ders 5 Genişletilmiş Notu: Facade, Flyweight, Proxy ve Chain of Responsibility Kalıpları

Ders 5 Genişletilmiş Notu: Facade, Flyweight, Proxy ve Chain of Responsibility Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Facade Kalıbı
- 2. Flyweight Kalıbı
- 3. Flyweight Örneği: Kimya Fabrikası Üretim Bilgi Sistemi
- 4. Proxy Kalıbı
- 5. Proxy Örneği: İzledikçe Öde Video Akışı
- 6. Davranışsal Kalıplara Giriş: Chain of Responsibility
- 7. Chain of Responsibility Örneği: Reklam Yönetim Yazılımı
- Örnek Sorular
- Dersin Toparlaması

1. Facade Kalıbı

Geçen hafta Adapter, Bridge, Composite ve Decorator kalıpları işlenmişti. Bu derse yapısal kalıplardan Facade ile devam edildi. Hoca dersin her zamankinden biraz kısa süreceğini, bu yüzden hızlı ilerleneceğini söyledi.

1.1 Adın anlamı

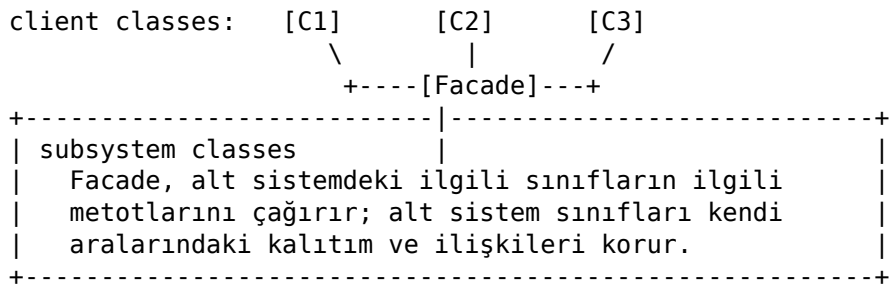
“Facade” sözcüğü Fransızca kökenlidir ve bir binanın dış cephesi, dış görünüşü anlamına gelir. Hoca Türkçedeki “faça” (yüz, dış görünüş) sözcüğünü hatırlattı ve bir dönem çok moda olan, evlerin dış cephesine yapılan “Amerikan kaplama” (siding) uygulamasını örnek verdi. İngilizcede de “facade of the house” denir. Kalıbın anlamı da budur: sisteme bir arayüz, bir erişim noktası vermek.

1.2 Amaç ve yapı

Slayttaki amaç şöyledir (ekrandan, 00:16:30): çeşitli alt sistemlerde bulunan farklı arayüzleri tek ve daha üst düzey bir arayüzde toplamak; böylece istekçiler alt sistemlerle ilgili alt düzey ayrıntılardan soyutlanabilir.

Dörtlü Çete’nin anlatmak istediği durum şudur: sistemin bir kısmında, bir sürü pakette bir sürü sınıf vardır. Son kullanıcının yapabileceği işlere ait sorumluluklar birçok farklı sınıfın birçok farklı metoduna dağılmıştır. Slayttaki örnekte yedi sınıflık bir yapıda üç dört sınıfın bazı metotları kullanılacaktır. İstekçiler bu sınıfların metotlarına farklı sıralarda ve farklı zamanlarda erişmek zorunda kalır; istekçi tarafındaki birçok sınıf ile alt sistemdeki birçok sınıf arasında karmaşık bir çoktan çoğa (many-to-many) ilişki oluşur.

Kalıbın önerisi, bu metotları bizim yazacağımız bir Facade sınıfının çağırması, istekçilerin de yalnız Facade sınıfı üzerinden sistemle ilişki kurmasıdır:



1.3 İç karmaşıklık ve refactoring

Hoca bu kalıbın iç karmaşıklığı düzeltmediğini, yalnız karmaşıklığın dışa yansımasını azalttığını vurguladı. Bu yüzden "aman aman" bir kalıp olmadığını, ama hızlı bir çözüm (quick and dirty fix) olarak işe yarayabileceğini söyledi.

Asıl yararını ileride işlenecek refactoring konusuyla bağladı. Refactoring'in tanımı, dışarıya verilen hizmetleri bozmadan iç yapıyı düzeltmektir. Bütüncül bir sistem üzerinde refactoring yaparken öyle bir noktaya gelinir ki, yapılacak tasarım değişikliği sistemi çok daha iyi bir hale getirecek ama dışarıya verilen hizmetlerin imzasını değiştireceği için testler bozulacaktır. Bu bir ikilemdir: geriye uyumluluk önemliyse değişiklik yapılmaz, değilse yapılır. Araya önceden bir Facade sınıfı konup sisteme ondan oluşturulan nesne üzerinden erişilirse, iç yapıdaki bir refactoring yalnız Facade sınıfını etkiler. Facade'ın istekçilere açık public metotlarının imzaları korunduğu sürece istekçi sınıflar hiç etkilenmez ve alt sistemin içinde istenen refactoring yapılabilir.

1.4 Ne zaman kullanılır

Slayttaki kullanım anları (ekrandan, 00:20:00):

- Karmaşık bir alt sisteme basit bir arayüz sağlanmak istenildiğinde,
- Alt sistemlerin çeşitli katmanlara ayrılması istenildiğinde,
- Bazı istekçilerin sistemin belli bir kısmıyla bağlaşımının azaltılması istenildiğinde.

Bağlaşımın azalmasını hoca kalıp şekli üzerinden saydı: alt sistemde dört ilgili sınıf ve üç istekçi varsa ve her istekçi her sınıfla ilişkiliyse $3 \times 4 = 12$ ilişkili, tam bağlı bir yapı olur. Araya Facade girince üç istekçi bir Facade ile, Facade da dört sınıfla ilişkide olur: $3 + 4 = 7$ ilişki.

Hoca bu basit kalıp için ayrıca kod yazmayacağını, yapının ne olacağının açıkça anlaşıldığını söyledi.

2. Flyweight Kalıbı

2.1 Adın anlamı ve amaç

Boks gibi sporlarda yarışmacılar ağırlıklarına göre en az hafif, orta ve ağır sıklet diye kategorilere ayrılır; en düşük kategoriye flyweight denir. Bu kalıpta da çok sayıda "hafif" nesneyle uğraşılır.

Kalıbın amacı, paylaşım yoluyla çok sayıda basit nesnenin kullanımını desteklemek ve bunu sistem kaynaklarını verimli kullanarak yapmaktır.

2.2 Metin düzenleyici örneği

Bir metin düzenleme programı yazıyoruz ve olabilecek en iyi çıktıyı istiyoruz: siyah harflerin beyaz arka planla keskinliği iyi olsun, harfler bulanık görünmesin. Düşük çözünürlüklü bir resmi büyütmeye kalkınca kutu kutu görüntüler oluşur; hocaya göre harfler de aslında birer resimdir (bitmap). Bu yüzden programda her harf ayrı bir grafik nesnesi olarak gösterilmek isteniyor.

Basitlik için sabit genişlikli bir yazı tipi (daktilo yazısı, kod yazarken kullanılan yazı tipleri gibi) düşünülebilir; böyle bir yazı tipinde örneğin "a" ile "ı" harfinin genişliği aynıdır. O zaman bir

A4 sayfaya en fazla belirli sayıda satır ve sütun yerleşir ve her satır-sütun konumunda bir harf bulunur. Değişken genişlikli yazı tipinde de metin yine N satır ve satır başına değişen sayıda sütundan oluşur.

Sorun şudur: her harf ayrı bir nesne olursa küçük bir belgede bile binlerce nesne olur ve belgenin bellekte ve diskte kapladığı yer çok artar. Hoca kendi hazırladığı 3 sayfalık bir ders öneri formunu saymış: boşluklarla birlikte yaklaşık 5000 harf. Her harfin bitmap'i 1 KB olsa:

$$5000 \times 1 \text{ KB} = 5000 \text{ KB} \approx 5 \text{ MB}$$

Yalnız metin içeren, başka hiçbir şey olmayan tek bir dosya için bu çok büyüktür.

2.3 Paylaşım fikri

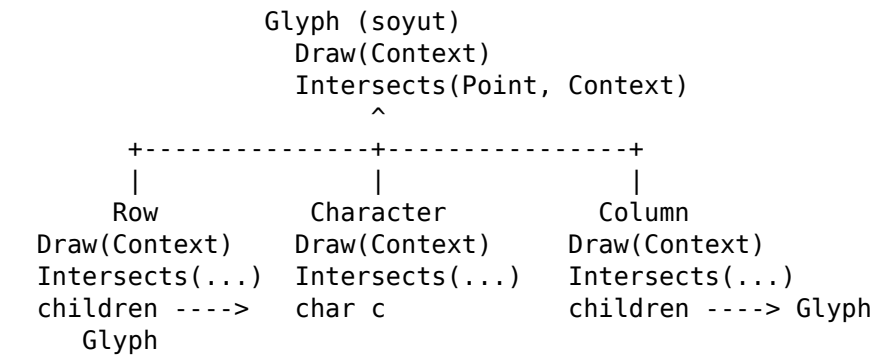
Çözüm, alfabedeki her harfi (büyük ve küçük harfler ayrı) tek bir harf nesnesi olarak ele almaktır. Örneğin "apparent" kelimesinde iki "a", iki "p", birer "r", "e", "n", "t" vardır; metin uzadıkça her harfin tekrar sayısı artar. Her satır ve sütunda yeni bir harf nesnesi oluşturmak yerine bütün harfler bellekte bilinen bir tabloda, bir havuzda yer alır ve her konumdan o tablodaki nesneye bir işaretçi verilir. Hoca işaretçinin kabaca 32 bit olacağını, bunun her harf için tekrarlanmasının bitmap saklamaya göre epey bellek tasarrufu sağladığını söyledi.

> **Ders sonrası not:** Hoca Unicode'daki karakter sayısını "1000 ya da 2000" diye tahmin etti ve emin olmadığını söyledi. Unicode'un güncel sürümleri büyük olasılıkla 140.000'den fazla karakter tanımlar. Örnek için önemli olan, metindeki harf sayısının alfabedeki farklı harf sayısından çok daha büyük olmasıdır.

Ortak havuzdan paylaşılan bu nesnelere flyweight denir. Metindeki her konum bir flyweight'e işaretçi tutar.

2.4 Örnek tasarım

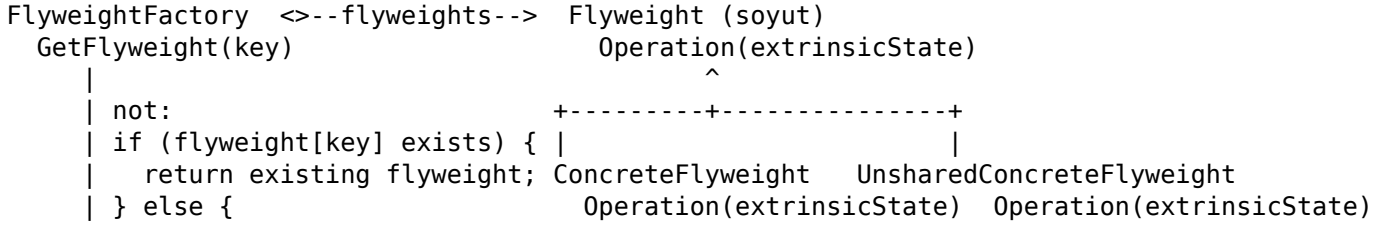
Bir metin, satırlar ve sütunlardan oluşan bir koleksiyon gibi düşünülebilir. Bu, Composite kalıbını hatırlatır ama farklı bir kalıptır. Satırları, sütunları ve karakterleri ortak modelleyen bir `Glyph` sınıfı önerilir; Composite'teki `Component` gibi (ekrandan, 00:30:00):



Bir sütun kendi içinde satırlar ya da karakterler içerebilir (ya da tersi). Bir satır çizileceği zaman içindeki her glif için çizim yaptırır; harf olan glif kendini belirli bir satır-sütun noktasına çizer. Her karakter nesnesinde hangi harfi içerdiğini saklayan bir üye alan vardır.

2.5 Kalıp yapısı

Slayttaki yapı metne dökülünce (ekrandan, 00:32:10):



```
     |   create new flyweight;        intrinsicState            allState
     |   add it to pool of flyweights;
     |   return the new flyweight;
     | }
Client ---> FlyweightFactory
Client ---> ConcreteFlyweight, UnsharedConcreteFlyweight
```

Havuzdaki belirli bir flyweight'e erişmek için bir anahtar gerekir. Anahtarı alıp ilgili nesneyi döndüren, yani havuza erişimi sağlayan sınıf `FlyweightFactory`'dir; `GetFlyweight(key)` metodu hangi harf isteniyorsa o anahtarla çağrılır.

`Flyweight` sınıfı soyuttur; yapılacak işlemler genelde alt sınıflarda çok farklı olacağından soyut metot olarak yazılır, ama alt sınıflara varsayılan bir davranış verilebiliyorsa somut metotlar da eklenebilir. Flyweight'ler genelde paylaşılır: metinde kaç küçük "a" varsa hepsi aynı küçük "a" nesnesini gösterir. Bazı flyweight'lerin paylaşılması istenmeyebilir; bu durumda `ConcreteFlyweight`'in kardeşi olarak `UnsharedConcreteFlyweight` sınıfı oluşturulur.

Fabrikanın önemi şudur: flyweight'ler paylaşılmazsa kalıbın ruhuna ters davranılmış ve bellek tasarrufu azalmış olur. Kalıp, paylaşılacak nesneleri üretmek için bir fabrika nesnesi oluşturulmasını özellikle önerir. `GetFlyweight` her zaman ilgili anahtara sahip aynı nesneyi döndürdüğü için ayrıca Singleton kalıbı kurmaya gerek yoktur.

İstenirse erişim daha da kısıtlanabilir: işlem metotları bir arayüzde tanımlanır ve somut flyweight sınıfı fabrikanın private iç sınıfı olarak kodlanır. Hoca benzer bir yolun Memento kalıbında da uygulanacağını, ama bunun şart olmadığını söyledi. Fabrikadan alınan nesne her seferinde klonlanabilir de; çok büyük bir sorun yoksa bu erişim kısıtlaması yapılmayabilir.

2.6 İçsel ve dışsal durum

Bir flyweight üzerinde işlem yapılırken nesnenin ihtiyaç duyduğu durum bilgisinin bir kısmı dışarıdan verilir, bir kısmı nesnenin içinde kalır. Metin örneğinde harf nesnesi kendi bitmap resmini bilir, ama hangi satır ve sütuna çizileceğini bilmez. Flyweight fabrikadan alındıktan sonra "kendini şu koordinatlara çiz" denir; çizme işlemi koordinatı parametre olarak alır, kendi içsel bilgisi olan harf resmini de kullanarak çizimi yapar.

- **Intrinsic state (içsel durum):** nesneye özgü, paylaşılan kısım. Harf nesnesinin resmi.
- **Extrinsic state (dışsal durum):** dışarıdan verilen kısım. Harfin hangi satır ve sütunda yer aldığı.

Kalıbın en çok tasarrufu sağlaması için ortak kullanılabilecek içsel bilginin olabildiğince geniş olması gerekir; içsel bilginin dışsal bilgiye oranı ne kadar büyükse tasarruf o kadar büyüktür. Kalıbı kullanmadan önce iki soruya evet denebilmelidir: durum bilgisi iki parçaya ayrılabiliyor mu ve ayrılan bilginin olabildiğince büyük kısmı ortak kullanılacak içsel bilgi olarak ayrılabiliyor mu?

Alternatif terimler olarak intrinsic yerine immutable, extrinsic yerine mutable da denir: içsel kısım değişmez, dışsal kısım dış etkilerle değişebilir. Bir tedbir olarak içsel üyeler `final` tanımlanabilir.

Nesne davranışı açısından, client bir flyweight ile çalışacağı zaman onu önce fabrikadan ister (ya da başka biri isteyip client'a vermiştir). Client somut flyweight nesneleri üzerinden çalışır ve onlara dışsal durumu verir. Flyweight nesnelerine olan başvuruları da client yönetir: metin örneğinde satırları saklayan fabrika değil, metin editörüdür.

2.7 Ne zaman kullanılır

Slayttaki koşulların tümünün doğru olması gerekir (ekrandan, 00:40:00):

- Bir uygulamanın çok fazla sayıda nesne kullandığı,
- Nesnelerin çok fazla olması nedeniyle saklama masraflarının çok yüksek olması,
- Nesnelerin durum bilgisinin çoğunun dışsallaştırılabilmesi,
- Dışsal durumları ayrılan nesnelerin bir çok grubunun yerini paylaşılabilecek az sayıda nesnenin alabileceği,
- Uygulamanın nesne kimliğine bağımlı olmadığı anlarda.

Biri bile doğru değilse ya kalıp kullanılamaz ya da getireceği kazanç yeterince büyük değildir. Hoca, çok az sayıda ama hacimli nesneyi flyweight'e ayırmanın mantıklı olmayacağını; orta sayıda orta ağırlıkta ya da çok sayıda hafif nesnenin değerlendirilebileceğini, günün sonunda orana bakılacağını söyledi.

2.8 Zayıf yön: nesne kimliği

> **Hocanın vurgusu:** Flyweight kalıbı nesne kimliğini (object identity) korumaz. Nesne kimliği sizin için sorunsa bu kalıbı kullanmamalısınız.

Bazı problemler nesne kimliğine çok bağlıdır. Hoca iki örnek verdi. Bir havayolu bir ayda yüz bin yolcu taşıdığını söylerse, bunların her birinin farklı kişi mi olduğu yoksa aynı kişinin birkaç kez mi taşındığı önemli bir istatistik olabilir. Bir reklam platformunda bir reklamı bir kişiye elli kez göstermenin etkisi ile elli kişiye birer kez göstermenin etkisi farklıdır; burada kişilerin kimliği önemlidir (kişiler nesne olsaydı TC kimlik numarası tekil tanımlayıcı olurdu). Flyweight'te bu tanımlayıcılık kaybolur: metinde bin tane "a" geçiyorsa her konumdaki "a" bellekteki aynı "a" nesnesini gösterir.

3. Flyweight Örneği: Kimya Fabrikası Üretim Bilgi Sistemi

3.1 Problem

Bir kimya fabrikası için üretim bilgi sistemi hazırlanıyor. Fabrikada birçok kimyasal madde üretilir ve bunların birleşiminden çok sayıda karışım hazırlanır; aynı madde birden fazla karışımda kullanılabilir. Hoca örneği "Design Patterns Java Workbook" kitabından aldığını söyledi.

Composite kalıbı akla gelebilir, ama aynı madde çok sayıda karışımda kullanıldığı için bu probleme uymaz. Farklı maddeler eklendiği için Decorator da pek uymaz.

3.2 İlk çözüm

İlk akla gelen tasarım (ekrandan, 00:50:00):

```
KimyasalMadde                                Karisim
- gram : double                              - isim : String
- isim : String                              - maddeler : LinkedList<KimyasalMadde>
- simge : String                             + setIsim( String )
- atomAgirligi : double            <>----    + getIsim( ) : String
+ Kimyasal( isim : String, simge : String,   + maddeEkle( KimyasalMadde )
            atomAg : double )                + tarifEt( ) : String
+ setGram( double )
+ getGram( ) : double
+ getIsim( ) : double
+ getAtomAgirligi( ) : double
+ getMol( ) : double     // return gram/atomAgirligi;
```

Maddenin adı (örneğin karbon), simgesi (C) ve atom ağırlığı (ondalıklı sayı) vardır. Atom ağırlığı örnekte doğrudan kullanılmasa da başka kimyasal hesaplamalar için saklanması gerekebileceği düşünülerek eklenmiştir. Gram dışındaki üç bilgi değişmez; bu yüzden kurucu bu üç parametreyi alır. Gram ise karışıma göre değişeceği için setter ve getter ile verilir. `getMol()` bir maddenin gram miktarını atom ağırlığına bölerek mol sayısını bulur.

`Karisim` sınıfında karışımın adı ve maddeler vardır. Karışımlarda farklı sayıda madde olacağından dizi yerine `LinkedList` kullanılır. `maddeEkle` karışıma madde ekler; aynı madde varsa `false`, yoksa `true` döndürecek biçimde de kodlanabilir. `tarifEt` "şu kadar gram şu madde" satırlarından oluşan bir metin döndürür.

Slayttaki örnek karışım: 100 g barutta 75 g potasyum nitrat, 15 g karbon ve 10 g kükürt bulunur.

`Karisim` kodu (ekrandan, 01:00:00):

```
package dp.flyweight.solution1;
import java.util.*;
```

```java
public class Karisim {
    private String isim;
    private LinkedList<KimyasalMadde> maddeler = new LinkedList<KimyasalMadde>();
    public String getIsim() { return isim; }
    public void setIsim(String isim) { this.isim = isim; }
    public void maddeEkle( KimyasalMadde madde ) {
        maddeler.add( madde );
    }
    public String tarifEt( ) {
        String tarif = isim;
        for( KimyasalMadde madde : maddeler )
            tarif += "\n" + madde.getGram() + "gr " + madde.getIsim();
        return tarif;
    }
}
```

3.3 İlk çözümün derste düzeltilmesi

`MainApp` çalıştırıldığında maddelerin gramları sıfır çıktı. Hoca kurucuya bakınca verilen sayıların gram değil atom ağırlığı olduğunu, `setGram` çağrısının kod yazılırken atlandığını gördü. İlk çözümü iki biçimde düzeltti:

- **Çözüm 1a:** `MainApp` içinde her madde nesnesi için `setGram` çağrılır (75, 15, 10), sonra `maddeEkle` ile karışıma eklenir.
- **Çözüm 1b:** `maddeEkle` metoduna ikinci parametre olarak `double gram` eklenir; metot içinde `madde.setGram(gram)` çağrılıp madde listeye eklenir.

Hoca bu sırada öğrencilerin aklına gelebilecek bir soruyu kendisi sordu: “Niye gramı parametre olarak vermiyorsunuz, iş daha kolay olurdu?” Cevabı çözüm 1b oldu. İki durumda da gram bilgisi maddeye dışarıdan verilir ve oluşturulmuş kimyasal madde nesnesinde saklanır; ilk çözümün beğenilmeme nedeni de budur.

3.4 İlk çözümün sorunu

Bir madde birden fazla karışımda yer alabilir; örneğin barutun içindeki karbon başka bir karışımda başka bir gramajla yer alır. İlk çözümde iki yol vardır:

- Karbonun her farklı gramajda yer aldığı karışım için ayrı bir karbon nesnesi oluşturulur. Ama ad, simge ve atom ağırlığı hiç değişmediği halde her kopyada tekrar saklanır ve bu baytlar boşa harcanır.
- Tek bir karbon nesnesi paylaşılır. Bu durumda gram bilgisi en son hangi karışıma eklendiyse onunkiyle ezilir.

Hoca ikinci durumu derste canlı gösterdi. `MainApp`’e ikinci bir karışım ekledi: kara baruttaki nitratın yerine öteki nitrat kullanılan ve hocanın “temiz barut” dediği bir karışım. Karbonu bu kez 14 g yaptı ve iki karışımda da aynı karbon nesnesi `c`’yi kullandı; her karışımdan sonra `c.getGram()` değerini yazdırdı. Konsol çıktısının görünen kısmı (ekrandan, 02:00:00):

```
Kara barut
75.0gr Sodyum Nitrat
15.0gr Karbon
10.0gr Kükürt
c.gr=15.0
Temiz barut
75.0gr Potasyum Nitrat
14.0gr Karbon
```

Hoca sonucu şöyle yorumladı: karbonun gramı 14’e düştü; aynı karbon nesnesinin farklı karışımlarda farklı gramajla yer aldığı bilgisinden yalnız en son içinde bulunduğu karışımdaki saklanıyor.

> **Ders sonrası not:** Slayttaki tarifte 100 g barutta 75 g potasyum nitrat vardır; derste çalıştırılan kodda ise “Kara barut” için sodyum nitrat, ikinci karışım için potasyum

nitrat kullanıldı. Klasik kara barut tarifinde büyük olasılıkla potasyum nitrat (KNO_3) kullanılır. Bu fark kalıbın anlatımını etkilemez.

Soru: İsim, simge ve atom ağırlığı static yapılsa sorun çözülür mü? **Cevap:** Hoca bu soruyu kendisi sordu ve cevapladı: belirli bir bellek tasarrufu sağlanır, ama aynı maddenin farklı örnekleri yalnız gram bilgisi farklı olarak yine birçok yerde bulunur. Bu yüzden static yapıp geçmek istenmez; daha akıllıca bir çözüm aranır.

3.5 İkinci çözüm: daha doğru modelleme

Temel ilkelerden biri "neyin değiştiğini bul ve onu sarmala"dır. Değişen gram bilgisidir; o halde gram bilgisi kimyasal madde sınıfından ayrılıp ayrı bir sınıfa alınmalıdır. Karışımın bileşenleri olur, her bileşende bir madde bulunur (ekrandan, 00:50:20):

```
Bilesen                                     Karisim
- gram : double                             - isim : String
- madde : KimyasalMadde       <------>      - bilesenler : LinkedList<Bilesen>
+ Bilesen( double, KimyasalMadde )          + setIsim( String )
+ getGram( ) : double                       + getIsim( ) : String
+ getMol( ) : double                        + bilesenEkle( madde : KimyasalMadde,
+ getMaddeAdi( ) : String                                  gram : double )
      | 1                                     (veya: bilesenEkle( Bilesen ))
      v 1                                   + tarifEt( ) : String
KimyasalMadde
- isim : String
- simge : String
- atomAgirligi : double
+ Kimyasal( isim : String, simge : String, atomAg : double )
+ getIsim( ) : double
+ getAtomAgirligi( ) : double
+ getMol( gram : double ) : double
```

`Bilesen` sınıfının kodu (ekrandan, 01:10:00):

```java
package dp.flyweight.solution2;

public class Bilesen {
    private double gram;
    private KimyasalMadde madde;

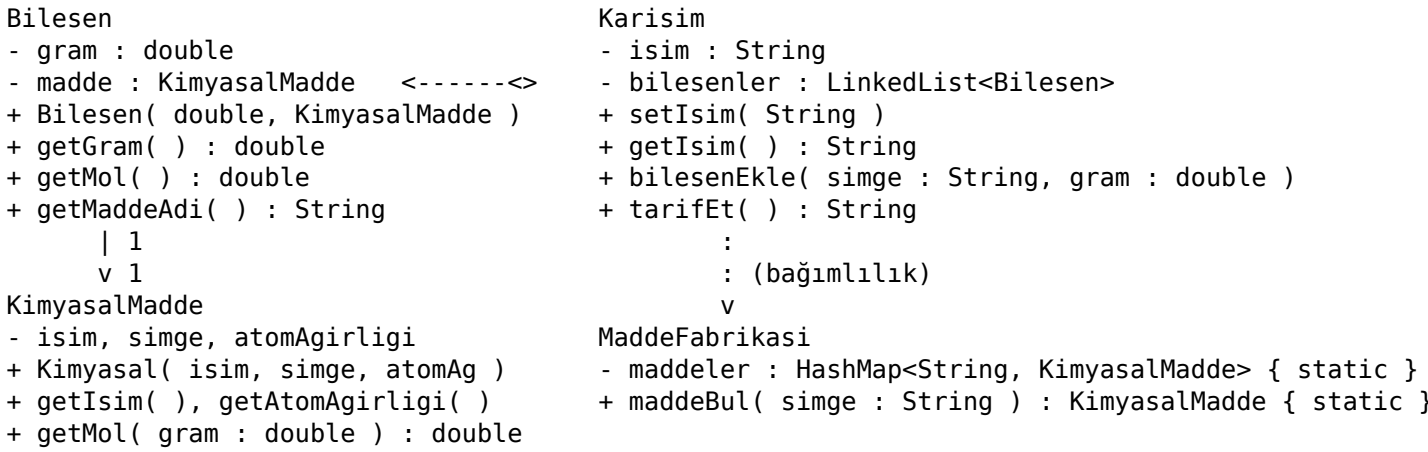
    public Bilesen( double gram, KimyasalMadde madde ) {
        this.gram = gram; this.madde = madde;
    }
    public double getGram( ) { return gram; }
    public double getMol( ) { return madde.getMol(gram); }
    public String getMaddeAdi( ) { return madde.getIsim( ); }
}
```

Bu tasarımda her bileşen bir kimyasal maddeyle birebir ilişkidedir. Aynı karbon nesnesi hangi karışımlarda yer alıyorsa hepsinde aynı nesne kimliğiyle yer almaya devam eder; yalnız gram bilgisi bileşenden bileşene değişir. Hoca bunun bir avantajını da gösterdi: karbonun simgesi yanlışlıkla "C" yerine "K" yazılmışsa (K potasyumun simgesidir), ilk tasarımda karbon içeren bütün karışımlara gidip simgeyi tek tek düzeltmek gerekirdi. Bu tasarımda tek karbon nesnesinin simgesi düzeltilince bütün karışımlar doğru tarif edilmeye devam eder.

Slayttaki kod derste karşılaştırılırken çözüm 1 ile çözüm 2'nin `Karisim` sınıflarının çok benzediği, `tarifEt`'in hemen hemen aynı kaldığı, yalnız madde yerine bileşen eklendiği görüldü.

3.6 Flyweight ile çözüm

İkinci tasarım fena değildir, ama artık gram bilgisi dışsal bilgi olarak dikkat çeker. Proje ilerledikçe gram gibi başka dışsal bilgiler de çıkabilir ve madde nesnelerinin sayısının çok olacağı öngörülür. Bellek tasarrufu için akla Flyweight gelir (ekrandan, 00:53:40):



Flyweight kalıbı ek olarak bir `MaddeFabrikasi` sınıfı getirir. Sistemde kayıtlı bütün maddeler bu fabrikadaki bir `HashMap` içinde yer alır ve `maddeBul` simgeye göre ilgili kimyasal maddeyi döndürür. Hoca işi kolay tutmak için bu üyeleri static tanımladığını söyledi; normalde static yapılmayabilir, çünkü farklı illerde ayrı fabrikalar varsa her biri için ayrı bir madde fabrikası nesnesi oluşturulabilir.

`Karisim.bilesenEkle` artık bileşen ya da madde nesnesi almaz; yalnız bileşenin simgesini ve gramını alır. Karışım, maddeyi `maddeBul` ile fabrikadan alır. Hoca, `Karisim` ile `KimyasalMadde` arasında da kesikli bir bağımlılık çizgisi çizmeyi denedi; araç çizgiyi düzgün çizmeye izin vermediği için vazgeçti ve iki sınıf arasında zaten `Bilesen` üzerinden dolaylı bir bağlaşım olduğunu söyledi.

Tasarımın geri kalanı değişmez: yalnız bileşen ekleme metodunun imzası değişir ve madde fabrikası eklenir. Ad, simge ve atom ağırlığı bilgileri ortak kullanıldığı için önemli ölçüde tasarruf edilir; her bileşende yalnız farklı gram bilgisi bulunur. Çözüm 2 ile çözüm 3 Eclipse'te karşılaştırıldığında (ekrandan, 01:20:00) `Bilesen` ve `KimyasalMadde` sınıflarında fark olmadığı, `Karisim`'da `bilesenEkle(KimyasalMadde, double)` yerine `bilesenEkle(String, double)` geldiği görüldü.

> **Hocanın vurgusu:** Bir veri en yakından hangi varlıkla ilişkiliyse o varlığın sınıfında saklanmalıdır; bu temel bir nesneye yönelim kavramıdır. Gram bilgisinin doğru yeri bileşendir. Gram madde içinde tutulursa, madde farklı bileşenlerde farklı gramajlarla yer aldığında belirsizlik oluşur.

4. Proxy Kalıbı

Moladan sonra son yapısal kalıp olan Proxy'ye geçildi. Hoca bununla yapısal kalıpların tamamlanacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** Dörtlü Çete'nin kitabında büyük olasılıkla yedi yapısal kalıp vardır: Adapter, Bridge, Composite, Decorator, Facade, Flyweight ve Proxy. Bu derste ve önceki derste bunların hepsi işlenmiş oldu.

4.1 Amaç ve Dörtlü Çete örneği

Proxy "vekil" demektir. Amaç, bir nesneye erişimi denetlemek için o nesnenin yerine geçecek başka bir nesneyi araya koymaktır. Bunun birçok farklı nedeni olabilir.

Dörtlü Çete'nin örneğinde ekrana bir resim çizmek, resim karmaşık ya da hacimliyse zaman alan bir işlemdir. Bu yüzden kelime işlem programları belgedeki resimleri ancak ilgili sayfa ekranda gösterileceği zaman yükleyip çizer. Hoca, yıllar önce resimli bir Word belgesinde hızla dolaşırken resimlerin yerinde, resmin eni ve boyu kadar, köşesinde kırmızı çarpı olan dikdörtgenler görüldüğünü hatırlattı. Geniş bant internetin yaygın olmadığı dönemde web sayfalarında da önce metin yüklenir, resimlerin yerinde benzer kırmızı çarpılı dikdörtgenler görülürdü; sayfa yüklenmesi bitince ya da sağ tıklayıp "yükle" denince resimler gelirdi. Hocaya göre o zamanlar Proxy'nin kullanımı gözle görülüyordu.

Bu tür programlar yanıt süresinin (response time) kısa olmasını ister. Belge açıldığında bütün resimler yüklenmez; açılan sayfanın resimleri yüklenir, kullanıcı gezinirken arka planda belki birkaç sayfa öncesi ve sonrasındaki resimler yüklenir. Belgenin sonundaki bir resim önceden yüklenmez.

Gerçek resim ile vekil resmin aynı biçimde kullanılabilmesi gerekir: ikisi de belirli bir satır ve sütuna çizilir, genişlik ve yükseklik gibi bilgileri vardır. Bu yüzden ortak bilgileri içeren tamamen soyut bir `Graphic` sınıfı (bütün metotları soyut olduğu için bir arayüz de olabilir) ve onun iki somut alt sınıfı vardır: asıl ilgilenilen `Image` ve ona vekalet eden `ImageProxy` (ekrandan, 01:46:40):

```
DocumentEditor -----> Graphic (soyut)
                        Draw()
                        GetExtent()
                        Store()
                        Load()
                           ^
              +------------+------------+
              |                         |
            Image   <------image------ ImageProxy
            Draw()                      Draw()
            GetExtent()                 GetExtent()
            Store()                     Store()
            Load()                      Load()
            imageImp                    fileName
            extent                      extent
```

```
ImageProxy.Draw():
    if (image == 0) {
        image = LoadImage(fileName);
    }
    image->Draw();

ImageProxy.GetExtent():
    if (image == 0) {
        return extent;
    } else {
        return image->GetExtent();
    }
```

Belge düzenleyici `Graphic` arayüzü üzerinden konuşur ve ilk aşamada proxy ile eşleşir. Proxy'nin çizme metodunda ilişkili resim yüklenmemişse (işaretçi sıfır, yani null) resim dosyadan yüklenir, sonra çizilir; yüklenmişse doğrudan çizilir. En-boy bilgisi istendiğinde resim yüklenmemişse proxy kendi `extent` bilgisini (ya da uygulamaya göre sıfır) döndürür, yüklenmişse resmin gerçek en-boyunu döndürür. Yani proxy yapılması gereken işi vekalet ettiği nesneye yaptırır.

Proxy gerekirse Decorator gibi çalışıp vekalet ettiği nesneden önce ya da sonra bazı işler yapabilir. Ama hocanın dikkat çektiği gibi burada Decorator'daki zincirleme ilişki yoktur; istekçi asıl nesneyle konuşmak yerine yalnız vekil üzerinden konuşur.

4.2 Kalıp yapısı ve bileşenler

Kalıp yapısında bir client, `Subject` üzerinden bir `Request` yapar. `RealSubject` ve `Proxy` aynı `Subject` üst sınıfının kardeş alt sınıflarıdır; proxy gerçek nesneyle bir sahiplik ilişkisi içindedir. Proxy kendine gelen her isteği asıl olarak `RealSubject`'e yaptırır, ama öncesinde ve sonrasında ek işlemler yapabilir.

Slayttaki bileşenler (ekrandan, 01:50:00):

- **RealSubject:** Erişimi denetlenecek veya oluşturulması uzun sürecek olan asıl nesne.
- **Proxy:** Asıl nesnenin yerine geçecek olan vekil.

- **Subject:** Vekilin aslının yerine geçebilmesi için vekil ve asıl nesneler için ortak bir arayüz sunar.

4.3 Ne zaman kullanılır

Slayttaki kullanım anları (ekrandan, 01:50:00):

- Herhangi bir nesneye basit bir işaretçiden daha fazlasının gerektiği her durumda kullanılabilir.
- Gereken yetenekler proxy sınıfına kodlanır.
- İşaret edilen nesne tamamen farklı bir adres uzayında bulunabilir.

Son maddeyi hoca şöyle açtı: vekil client ile aynı bilgisayarda, vekalet ettiği nesne ise başka bir bilgisayarda olabilir. Dağıtık hesaplama yapan sistemlerde bu mantık çok kullanılır. Hesaplama açısından ağır bir işin alt hesaplamaları ağ üzerinden iletişim kuran farklı bilgisayarlarda yapılır; istekçi isteği yalnız aradaki bir bilgisayara gönderir, o bilgisayar alt problemleri diğerlerine dağıtır, sonuçları birleştirip istekçiye gönderir. Özellikle bilimsel hesaplamalarda bu yoğun olarak yapılır.

5. Proxy Örneği: İzledikçe Öde Video Akışı

5.1 Problem

Hoca bu örneği bir kaynaktan aldığını ama kaynağı not etmediğini söyledi. Elimizde bir izledikçe öde (pay per view) video sunucusu var. Hocaya göre bu tür platformlar pandemi döneminde ve sonrasında yaygınlaştı. Abonelik sahibi olmayan biri istediği videoyu indirememelidir. Benzer biçimde bazı büyük otellerde özel film kanallarında izlenen filmler, minibardan alınan ücretli içecek gibi oda faturasına yansıtılırdı.

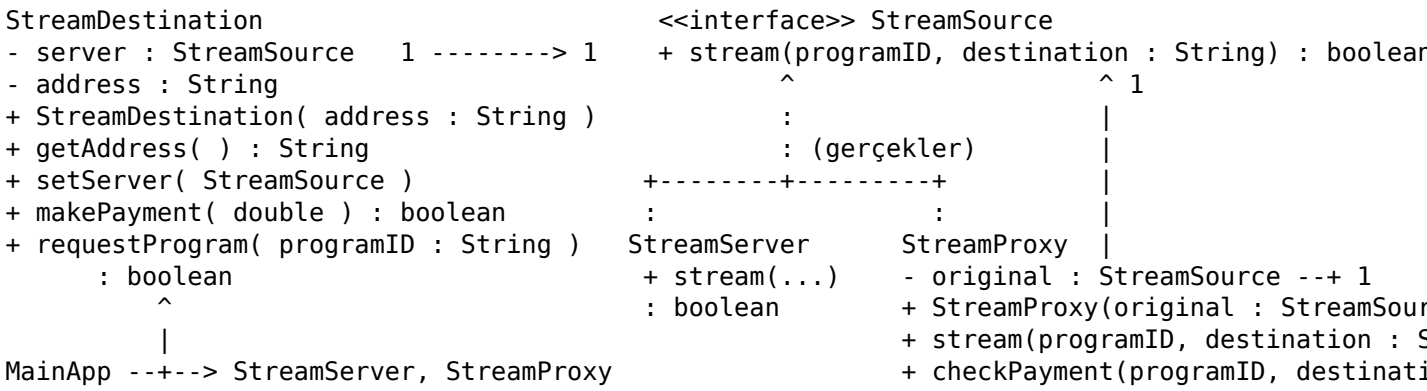
Elimizdeki video sunucusu çok basittir: yalnız belirli bir görüntüyü belirli bir adrese aktarır (stream). Abonelik, ödeme ya da yaş kategorisi (örneğin 7, 13, 18 yaş; belirli kategorilerde PIN girilmesi) gibi hiçbir erişim denetimini desteklemez. Yepyeni bir çözüm satın almak yerine mevcut çözümün bu işlevleri destekleyecek biçimde geliştirilmesi isteniyor. Üstelik sunucu yazılımının kaynak koduna erişilemiyor. Akla araya bir vekil koymak gelir.

İstekçinin ilgili nesneye erişim hakkı olup olmadığını denetleyen, varsa isteği ileten, yoksa iletmeyen bu tür vekile **Protection Proxy** denir. Başka proxy türleri de vardır; hoca, çalışılan nesne başka bir bilgisayardaysa vekile stub da dendiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Dörtlü Çete büyük olasılıkla dört proxy türü sayar: başka adres uzayındaki nesneyi temsil eden remote proxy, pahalı nesneyi gerektiğinde oluşturan virtual proxy (resim örneği), erişimi denetleyen protection proxy ve erişim sırasında ek işler yapan smart reference. Java RMI gibi teknolojilerdeki stub'lar remote proxy örneğidir.

5.2 Tasarım

Slayttaki çözüm (ekrandan, 01:57:15):



`StreamServer`'ın tek yapabildiği, belirli kimlikli bir programı belirli bir hedefe göndermektir. Hoca bu sunucunun büyük olasılıkla `StreamSource` arayüzünü zaten gerçeklediğini varsaydı: ücretli, karmaşık bir yazılımın API noktaları olarak böyle arayüzler sunması beklenir. Gerçeklemiyorsa ve kaynak koduna erişilebiliyorsa bu arayüz eklenir.

`StreamProxy` de bu arayüzü gerçekler ve bir orijinal `StreamSource` ile birebir ilişkidedir. Hoca diyagramda bu birebir ilişkiyi işaretledi. Dörtlü Çete'nin kalıp şemasında proxy doğrudan vekalet ettiği gerçek nesneye bağlıdır; bu örnekte ise ortak arayüze bağlanmıştır. İkisi de olabilir; arayüze bağlamak biraz daha esnektir, çünkü proxy farklı türde akış sunucularına da vekalet edebilir.

`StreamDestination` hedefi temsil eder: bir otel odası numarası, bir akış platformundaki üye hesabı ya da kullanılan bilgisayarın veya set üstü kutunun IP adresi gibi bir adres. İstemciler akış kaynaklarıyla doğrudan değil, `StreamSource` arayüzü üzerinden proxy ile konuşur.

5.3 Kod

`StreamProxy` sınıfı (ekrandan, 02:00:45):

```
package dp.proxy.example;

public class StreamProxy implements StreamSource {
    private StreamSource original;

    public StreamProxy(StreamSource original) {
        this.original = original;
    }

    public boolean stream(String programID, String destination) {
        if( checkPayment(programID, destination) )
            return original.stream(programID, destination);
        else
            return false;
    }
    ...
}
```

Bir akış isteği geldiğinde proxy önce ödemenin yapılıp yapılmadığını kendi `checkPayment` metoduyla denetler; bu metot duruma göre true ya da false döndürür. Denetimden geçilirse orijinal sunucuya programı ilgili hedefe göndermesi söylenir; geçilmezse hiçbir şey gönderilmeden `false` döndürülür. Böylece bir Protection Proxy gerçeklenmiş olur. Ödemenin nasıl denetleneceği bilinmediği için hoca yalnız iskelet kod yazdığını söyledi.

`StreamDestination`'da sunucu ve adres kurucuyla ya da setter ile verilir. `makePayment` gerçekten ödeme yapmak için gereken işlerin yapılacağı metottur. `requestProgram` belirli bir programın erişim isteğini ilişkili sunucuya, yani aslında proxy'ye yönlendirir. `MainApp` bir gerçek sunucu, bir proxy ve bir istemci oluşturur, istemciyi proxy ile ilişkilendirir ve istemci belirli kimlikli programları ister.

5.4 Proxy ile Adapter farkı

> **Hocanın vurgusu:** Proxy ile Adapter birbirine benzer; ikisinde de bir istek başka bir tarafa iletilir. Ama amaçları farklıdır. Adapter'da yalnız uyumsuz iki metot imzasının birbirine uyuşturulması vardır, başka ekstra hiçbir şey yoktur. Proxy ise araya erişim denetimi gibi ek işler koyar. Bu farkı unutmayın.

Örnekte `StreamServer` `StreamSource` arayüzünü gerçeklemiyorsa ve kaynak koduna da erişilemiyorsa Proxy kurulamaz; bu durumda Adapter yapılır. Proxy'nin somut sunucu yerine ortak arayüze bağlanması onu Adapter'dan biraz daha uzaklaştırır; birebir sahiplik ilişkisi ise Adapter'a benzerliği artırır. Hocaya göre sonuçta bunlar farklı işler yapan farklı kalıplardır.

6. Davranışsal Kalıplara Giriş: Chain of Responsibility

Yapısal kalıplar bitince davranışsal (behavioral) kalıplara geçildi. İlk davranışsal kalıp Chain of Responsibility'dir (sorumluluk zinciri).

6.1 Amaç

Kalıp, birbiriyle ilişkili bir nesne zincirinin ya da hiyerarşisinin zaten var olduğu durumlarda geçerlidir. Zincirin bir halkası kendisine gelen bir isteği karşılayamıyorsa ya da isteği daha iyi karşılayabilecek biri varsa, isteğin o noktaya iletilmesini sağlar.

Hocanın günlük hayat örneği: bir mağazaya ürün iade edeceksiniz; çalışanın yetkisi yoktur ve mağaza müdürünü çağırır. İade daha çetrefilli bir durumsa mağaza müdürünün de yetkisi olmayabilir ve bölge müdürünü telefonla arar.

İlk alıcının mesajı yanıtlayamaması, ilgili metoda sahip olmamasından kaynaklanmaz. O nesnenin metodu vardır; ama isteği yürütecek yetkisi ya da en güncel bilgisi olmayabilir.

6.2 Dörtlü Çete örneği: bağlama duyarlı yardım

Hoca eski ofis programlarındaki bir yardım biçimini anlattı. Bir iletişim kutusunda (örneğin yazı tipi kutusu) bir bileşenin işlevini anlamadığınızda yardım düğmesine tıklarsınız; eski sürümlerde fare imleci yanında soru işareti olan bir hale gelirdi. Bu haldeyken bir yere tıklayınca tıklanan alana odaklanmış yardım gösterilirdi: sekmenin başına tıklanırsa bütün sekmeyle ilgili ayrıntılı yardım, "efektler" kısmına tıklanırsa yalnız o kısımla ilgili yardım. Hoca odaklanmış yardımı daha iyi bulduğunu söyledi.

Her bileşenin yardım bilgisi kodlanmamış olabilir. Kodlanmışsa gösterilir; kodlanmamışsa bileşenin içinde bulunduğu üst bileşenin yardımı gösterilir; onun da yoksa bir üstününki. Bu iletim en iyi yardımı verebilecek yer bulunana ya da zincirin sonuna gelinene kadar sürer. Bunun için her bileşenin hangi üst bileşenin içinde olduğunu bilmesi gerekir. Kalıp, zaten böyle bir ilişki içinde olan varlıklar arasında kurulur; böyle doğal bir ilişki yoksa ya bu kalıp seçilmez ya da ilişki zinciri ayrıca kurulur.

Nesnelerin örnek durumu (ekrandan, 02:10:00): OK ve Print düğmeleri bir `PrintDialog` içinde bulunur.

```
aPrintButton.handler --+
                       +--> aPrintDialog.handler --+
anOKButton.handler ----+                           +--> anApplication.handler
                            aSaveDialog.handler ---+

specific  ---------------------------------------------------->  general
```

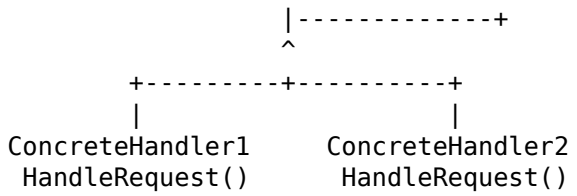
Print düğmesinde yardıma basıldığında düğme yardımı kendisi gösterebiliyorsa gösterir; gösteremiyorsa `handler`'ı, yani içinde bulunduğu nesnenin yardım metodunu çağırır. O nesne de yardım verebiliyorsa verir, veremiyorsa isteği uygulamanın yardım dosyasına iletir. Ağacın uçları daha belirli ve ayrıntılı, üst kısımları daha genel ve daha az ayrıntılıdır.

Bunun için ortak bir üst soyut sınıf ya da arayüz gerekir; mesajı üst bileşene iletmek için de bu arayüzde ayrı bir metot tanımlı olmalıdır. Aksi halde "üst bileşenimin türü ne, ona göre onun şu metodunu çağır" diye switch-case ya da if-else zinciri kurmak gerekir; hoca bunun iyi bir kodlama biçimi olmadığının önceki haftalarda konuşulduğunu hatırlattı. İlişkiyi en üst sınıf düzeyinde, tercihen hiyerarşinin kökünde tutmak gerekir; o zaman hiyerarşinin bütün bileşenleri birbirine zincirlenebilir. Slaytta bunun için bir `HelpHandler` soyut sınıfı ve sequence diyagramı da vardı: Print düğmesi yardım edemiyorsa `PrintDialog`'a, o da edemiyorsa `Application`'a iletir.

6.3 Kalıp yapısı

Slayttaki yapı (ekrandan, 02:13:10):

```
Client -------> Handler (soyut)  <----+
                  HandleRequest()     | successor
```




```
Nesneler:
aClient.aHandler --> aConcreteHandler.successor --> aConcreteHandler.successor
```

Kalıp bileşenleri:

- **Client:** Yanıtlanacak mesajın kaynağı.
- **Handler:** Yanıtlanacak mesajı tanımlayan arayüz. Zincir bağlantısını (successor) da soyut sınıf olarak gerçekleyebilir.
- **ConcreteHandlerX:** Mesajı yanıtlar veya bağlantılı olduğu bir başkasına iletir.

Client bir istekte bulunduğunda, isteği karşılayabilecek nesnelerle soyut sınıf üzerinden ilişkidedir ve hangi somut handler verildiyse isteği ona yöneltir. O karşılayabiliyorsa ne âlâ; karşılayamıyorsa üst sınıftan kalıtımla gelen successor üzerinden isteği bağlı olduğu nesneye iletir. Successor'ı set ve get eden metotlar, alt sınıflarda aynı biçimde çalışıyorsa soyut `Handler`'a konmalıdır. Bazı kısımları ortak bazı kısımları farklıysa ileride görülecek Template Method kalıbı yardımcı olabilir. Somut handler'lardan kalıtımla daha özelleşmiş handler'lar da türetilebilir; kalıp buna izin verir.

6.4 Ne zaman kullanılır ve zayıf yönü

Bir metot çağrısını yanıtlayabilecek birden fazla nesne varsa ve en iyi yanıtı verecek (ya da yanıt verebilecek tek) nesne önceden bilinemiyorsa bu kalıp kurulur. Kalıbın bir güzelliği de ilişkili nesneler zincirinin çalışma anında kurulabilmesi ve değiştirilebilmesidir: "şu bileşeni şunun üstüne ya da altına al" gibi metotlarla zincir istendiği gibi düzenlenebilir.

> **Hocanın vurgusu:** Kalıbın en dikkat edilmesi gereken zayıf noktası, ilişki zincirinin doğru yapılandırılması gerekliliğidir. Aksi halde metot çağrısının gerçekten yanıtlanacağının hiçbir garantisi kalmaz. Buna çok dikkat edin.

Hoca bunu bürokrasisi yoğun kurumlara benzetti: bir iş için bir departmana gidersiniz, "önce şuraya git" denir; oraya gidersiniz, "en başta gittiğin yere git" denir. Zincir yanlış kurulursa kalıp da böyle davranır. Dikkat edildiği takdirde yararlı bir kalıptır. Zincir kurulu değilse onu kuracak kodu da bizim eklememiz gerekir.

7. Chain of Responsibility Örneği: Reklam Yönetim Yazılımı

7.1 Problem

Bu örneği hoca kendisi oluşturmuş (slaytta "Kaynak: YES"). Bir reklam yönetim yazılımı hazırlanıyor. Portföyde farklı konularda web siteleri var ve reklamların en büyük etkiye ulaşması için hedef kitlenin uğradığı sitelerde gösterilmesi isteniyor. Slayttaki kampanyalar (ekrandan, 02:20:00):

- **Kampanya 1:** Spor sitesinin 18-39 yaş arası erkek ziyaretçilerine tıraş bıçağı reklamını göster, 18-39 yaş arası kadın ziyaretçilerine fitness salonu reklamını göster, diğer ziyaretçilere spor mağazasındaki indirimin reklamını göster.
- **Kampanya 2:** Haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamını göster, aksi halde spor sitesinin kampanyasını aynen uygula.
- **Kampanya 3:** Ziyaretçinin coğrafi bölgesi ve gelir düzeyine göre portföydeki lokanta reklamlarından birini göster.

7.2 Neden Decorator değil

Kampanya 2, Kampanya 1'e ek olarak bir ön adım içerir; geri kalan her şey aynıdır. Bu yüzden ilk bakışta problem Decorator ile çözülebilir gibi görünür. Ama Decorator'da tek bir varlık birden fazla yeteneğe sahip olur ve bir iş yapıldığında bütün o yetenekler birlikte devreye

girer. Bu kampanyalarda ise birbiriyle ilişkili yeteneklerin hepsi kullanılmaz; doğrusal biçimde ilişkili olası seçenekler arasından yalnız en uygunu bulunup çalıştırılır.

> **Hocanın vurgusu:** Kampanya 1 ile Kampanya 2'nin birbirine başvurmasına bakıp bunu Decorator olarak düşünmeyin. Burada bir varlığa yeni yetenekler kazandırılmıyor; olası seçeneklerin en uygunu aranıyor. Bu yüzden problem Chain of Responsibility ile çözülür.

7.3 Tasarım

Ana uygulama reklam platformudur ve kayıtlı kullanıcıların cinsiyet, gelir düzeyi ve yaş bilgileri tutulur. `User` sınıfının kurucusu bütün bilgileri alır; getter'ları ve `toString` metodu vardır. Yaş aralığı sorgusu en uygun yer olan bu sınıfta kodlanır: `yasDahilMi(alt, ust)` kullanıcının yaşı aralıktaysa true, değilse false döndürür. Gelir düzeyi (A, B, C) ve cinsiyet (kadın, erkek) bu değerlerin dışında değer alamayacağı için enum olarak tanımlanmıştır.

Asıl zincir reklam tarafında kurulur. Zincirin bir ucundan gelen kullanıcının zincirin hangi halkasına uygun olduğu bulunur ve yalnız o halka reklamını gösterir. Bütün reklamların ortak soyut üst sınıfı `Reklam`'dır; reklamı gösterecek bir `Banner` üyesi vardır (gerçekte resim çizilmez, yalnız sınıf adı yazdırılır). Zinciri saklayan üye, `Reklam` türündeki `sonraki` üyesidir; hem `Banner` hem `sonraki` kurucuda verilir.

`Reklam`'ın iki soyut metodu vardır: `reklamGoster` (reklam farklı reklamlarda farklı biçimde gösterilebilir) ve `reklamUygunMu` (hangi yaş, gelir ya da cinsiyetteki kullanıcıya uygun olduğu alt sınıfa göre çok farklı değişeceği için üst sınıfta bir şey söylenemez). Somut metot `reklamKotar` ise zinciri yönetir (ekrandan, 02:30:00):

```
public abstract boolean reklamUygunMu ( User u );
public abstract void reklamGoster( User u );
public final void reklamKotar( User u ) {
    if( reklamUygunMu(u) ) {
        System.out.println( this.getClass().getName()
            + "->" + u );
        reklamGoster(u);
    }
    else if( sonraki != null ) {
        System.out.println( this.getClass().getName()
            + "->" + sonraki.getClass().getName());
        sonraki.reklamKotar(u);
    }
}
```

Reklam önce kendisinin bu kullanıcıya uygun olup olmadığına bakar. Uygunsa kendi sınıf adını ve kullanıcıyı yazdırır, yani "bu reklamı bu kullanıcıya gösterdim" der, sonra reklamı gösterir. Uygun değilse ve sonraki reklam null değilse, kendi sınıf adını ve sonraki reklamın sınıf adını yazdırır, yani "kullanıcıyı şu reklama yönlendirdim" der, sonra isteği `sonraki.reklamKotar(u)` ile iletir. Yazdırma satırları akışın nasıl yürüdüğünü göstermek için konmuştur.

Somut reklamlar (`TrasBicak`, `FitnessSalonu`, `IndirimSporMagaza`, `OtoDergi`) `Reklam`'dan türer, `Banner` ve sonraki reklam bilgisini üst sınıfın kurucusuna iletir. `reklamGoster` gövdeleri boştur. Örneğin `OtoDergi.reklamUygunMu`, kullanıcının cinsiyeti erkek, gelir düzeyi A ve yaşı 18-29 arasındaysa true döndürür; `TrasBicak.reklamUygunMu` cinsiyet erkek ve yaş 18-39 arasındaysa true döndürür. Başka kampanyalar tanımlanırsa onların reklamları da `Reklam`'ın kardeş alt sınıfları olarak eklenir.

7.4 Zincirin doğru kurulması

Zincir yanlış kurulursa ve zincirdeki hiçbir reklam kullanıcıya uygun değilse istek, `sonraki`'nin null olduğu yere kadar gider ve kullanıcıya hiç reklam gösterilmez. Bu kodda bir halkanın sonraki halkanın kullanıcıya uygun olup olmadığını sorgulaması kodlanmamıştır. Bu yüzden zincirin sonunda her türlü kullanıcıya hizmet verebilecek bir reklam olmalıdır.

Hoca alternatifi de anlattı: bir düğüm, isteği iletmeden önce sonrakinin yanıt verip veremeyeceğine bakabilir; o vermese bile onun bağlı olduğu bir başkası verebilir. Sorgu bütün zincirde dolaşıp özyinelemeli olarak geri döner; kimse almamışsa "madem bana geldi, bu reklamı ben vereyim" denebilir. Ama bu işi epey karmaşıklaştırır; zincirin doğru kurulduğundan emin olmak çok daha kolaydır.

7.5 Ana uygulama ve çalıştırma

`MainApp` farklı özelliklerde üç kullanıcı tanımlar (ekrandan, 02:36:05):

```
users = new User[3];
users[0] = new User( Cinsiyet.ERKEK, GelirDuzeyi.C, 23 );
users[1] = new User( Cinsiyet.ERKEK, GelirDuzeyi.A, 53 );
users[2] = new User( Cinsiyet.KADIN, GelirDuzeyi.B, 33 );
```

Her kampanya ayrı bir metotta tanımlanır. Kampanya 1'de her kullanıcıya uygun olan reklam spor mağazası indirim reklamıdır. Hoca zinciri bağlı liste yerine daha basit kod için dizi ile kurdu: zincirin başı dizinin sıfırıncı elemanı, sonu son elemanıdır ve herkese uygun reklam en sona, sonraki reklamı olmadan konur. Diziyi sondan başa doğru (2, 1, 0) doldurmayı tercih ettiğini, bunun büyük olasılıkla zincirin sonunu vurgulamak için olduğunu söyledi. Kampanya 1'in zincir sırası tıraş bıçağı, fitness salonu, spor mağazası indirimidir; kampanya üç kullanıcıya da uygulanır.

Kampanya 2'nin kodu (ekrandan, 02:40:00):

```
public void kampanya2( ) {
    Reklam[] reklamlar = new Reklam[4];
    reklamlar[3] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[2] = new FitnessSalonu(new Banner(), reklamlar[3]);
    reklamlar[1] = new TrasBicak(new Banner(), reklamlar[2]);
    reklamlar[0] = new OtoDergi(new Banner(), reklamlar[1]);
    reklamlar[0].reklamKotar(users[0]);
    reklamlar[0].reklamKotar(users[1]);
    reklamlar[0].reklamKotar(users[2]);
}
public static void main(String[] args) {
    MainApp app = new MainApp();
    app.kampanya1();
    app.kampanya2();
}
```

Hoca Kampanya 2'yi elle yeniden kodladığını, aslında Kampanya 1'in dizisinin başına bir eleman eklemek gibi bir yol izlenebileceğini söyledi. Ama kampanyaları birbirinden bağımsız tutmayı tercih etti; Kampanya 2'nin Kampanya 1'in reklamlarına erişmesi hem doğru olmayacak hem de daha fazla kod gerektirecekti. Bu bir miktar kod tekrarı yaratır, ama müşteri gereksinimleriyle uyumsuz değildir.

Kampanya 1'in çalıştırılmasında beklenen sonuç: 23 yaşındaki erkek kullanıcıya tıraş bıçağı; 53 yaşındaki erkek kullanıcıya yaşı nedeniyle diğer kategorilere uymadığı için spor mağazası indirimi; yaş aralığına giren kadın kullanıcıya fitness salonu reklamı. Konsol çıktısı (ekrandan, 02:36:05):

```
dp.chainOfResponsibility.example.TrasBicak->Erkek,C,23
dp.chainOfResponsibility.example.TrasBicak->dp.chainOfResponsibility.example.FitnessSalonu
dp.chainOfResponsibility.example.FitnessSalonu->dp.chainOfResponsibility.example.IndirimSporMa
dp.chainOfResponsibility.example.IndirimSporMagaza->Erkek,A,53
dp.chainOfResponsibility.example.TrasBicak->dp.chainOfResponsibility.example.FitnessSalonu
dp.chainOfResponsibility.example.FitnessSalonu->Kadın,B,33
```

Okun sağında kullanıcı bilgisi yazıyorsa o reklam o kullanıcıya gösterilmiştir; okun sağında başka bir reklam sınıfı yazıyorsa kullanıcı o reklama yönlendirilmiştir.

Kampanya 2 çalıştırıldığında da sonuç aynı çıktı: 23 yaşındaki erkeğe tıraş bıçağı. Zincirin başına konan otomobil dergisi reklamı hiçbir kullanıcıya gösterilmedi, çünkü 18-29 yaş arası A

gelir grubu erkek kullanıcı yoktur. Hoca, gösterseydi zinciri yanlış kurmuş ya da gereksinimleri yanlış anlamış olabileceğini, göstermemesinin doğru olduğunu söyledi.

7.6 Template Method notu

Hoca son olarak, `Reklam` soyut sınıfı ile somut alt sınıfları arasında aslında bir Template Method kalıbı kurulduğunu söyledi (`reklamKotar` final bir şablon metot, `reklamUygunMu` ve `reklamGoster` alt sınıfların doldurduğu soyut adımlardır). Slayttaki not (ekrandan, 02:40:00): "Reklam ile somut sınıflar arasında Template Method kalıbı kurulu. Kitap sırasına geçince bu kalıbı henüz görmemiş olduk, ileride göreceğiz." Template Method işlenirken kendi örneği de yapılacak.

Örnek Sorular

1. (derste sözlü, 00:20:00) Bir alt sistemde istekçilerin kullandığı dört sınıf ve üç istekçi sınıf vardır. Her istekçi her sınıfla ilişkiliyse kaç ilişki olur? Araya bir Facade konursa kaç ilişki kalır?

> **Çözüm (derste verildi):** Tam bağlı yapıda $3 \times 4 = 12$ ilişki vardır. Facade ile üç istekçi Facade'a, Facade da dört sınıfa bağlanır: $3 + 4 = 7$ ilişki.

2. (derste sözlü, 00:26:15) Her harf ayrı bir bitmap nesnesi olarak saklanırsa ve her harfin bitmap'i 1 KB ise, boşluklarla yaklaşık 5000 harf içeren 3 sayfalık bir belge ne kadar yer kaplar? Flyweight bunu nasıl azaltır?

> **Çözüm (derste verildi):** 5000×1 KB $= 5000$ KB, yaklaşık 5 MB (python ile doğrulandı: $5000 \times 1024 = 5\,120\,000$ bayt, yaklaşık 4,9 MiB). Flyweight ile alfabedeki her harf (büyük ve küçük ayrı) bir kez saklanır, metindeki her konum yalnız ilgili harf nesnesine bir işaretçi (hocaya göre kabaca 32 bit) tutar. Konum bilgisi dışsal olarak çizim sırasında verilir.

3. (ekrandan, 00:40:00) Flyweight kalıbının kullanılabilmesi için hangi koşullar sağlanmalıdır? Kalıp hangi tür problemlerde kullanılmamalıdır?

> **Çözüm (derste verildi):** Koşulların tümü doğru olmalıdır: uygulama çok sayıda nesne kullanır; nesnelerin çokluğu nedeniyle saklama masrafı yüksektir; nesnelerin durum bilgisinin çoğu dışsallaştırılabilir; dışsal durumları ayrılan nesnelerin büyük bir grubunun yerini paylaşılabilecek az sayıda nesne alabilir; uygulama nesne kimliğine bağımlı değildir. Nesne kimliğinin önemli olduğu problemlerde (bir havayolunun taşıdığı yolcuların farklı kişiler olup olmadığı, bir reklamın kaç farklı kişiye gösterildiği) kullanılmamalıdır, çünkü paylaşılan nesnelerde kimlik kaybolur.

4. (derste sözlü, 00:49:01) Kimya örneğinin ilk çözümünde maddenin isim, simge ve atom ağırlığı üyeleri static yapılırsa sorun çözülür mü? Doğru çözüm nedir?

> **Çözüm (derste verildi):** Hayır. Static üyeler bir miktar bellek tasarrufu sağlar, ama aynı maddenin yalnız gram bilgisi farklı olan örnekleri yine birçok yerde bulunur. "Neyin değiştiğini bul ve onu sarmala" ilkesine göre değişen gram bilgisi `KimyasalMadde`'den ayrılıp `Bilesen` sınıfına alınır. Sonra maddelerin paylaşımı `MaddeFabrikasi.maddeBul(simge)` ile Flyweight kalıbına bağlanır.

5. (derste çalıştırıldı, 01:17:43; ekrandan, 02:00:00) İlk çözümde aynı karbon nesnesi önce kara baruta 15 g, sonra ikinci bir barut karışımına 14 g olarak eklenirse karbonun `getGram()` değeri ne olur? Bu neyi gösterir?

> **Çözüm (derste verildi):** Kara barut eklendikten sonra `c.gr=15.0` yazdırılır; ikinci karışıma eklendikten sonra karbonun gramı 14 olur. Gram bilgisi madde nesnesinde tutulduğu için madde yalnız en son eklendiği karışımın gramını taşır, önceki karışımın bilgisi kaybolur. Gram bilgisinin doğru yeri bileşendir.

6. (derste sözlü, 01:55:24) Video akışı örneğinde `StreamServer`, `StreamSource` arayüzünü gerçeklemiyorsa ve kaynak koduna erişilemiyorsa ne yapılır? Proxy ile Adapter arasındaki fark nedir?

> **Çözüm (derste verildi):** Bu durumda Proxy kurulamaz; Adapter yapılır. Adapter yalnız uyumsuz iki metot imzasını birbirine uyuşturur, başka ek iş yapmaz. Proxy ise

ortak arayüzü gerçekler ve isteği asıl nesneye iletmeden önce ya da sonra erişim denetimi (örnekte ödeme denetimi) gibi ek işler yapar.

7. (ekrandan, 02:36:05 ve 02:40:00) Kullanıcılar (Erkek, C, 23), (Erkek, A, 53) ve (Kadın, B, 33) olsun. Kampanya 1 zinciri (tıraş bıçağı → fitness salonu → spor mağazası indirimi) ve Kampanya 2 zinciri (otomobil dergisi → tıraş bıçağı → fitness salonu → spor mağazası indirimi) ile her kullanıcıya hangi reklam gösterilir?

> **Çözüm (derste verildi):** İki kampanyada da sonuç aynıdır: 23 yaşındaki erkeğe tıraş bıçağı, 53 yaşındaki erkeğe (yaşı 18-39 aralığı dışında olduğu için) spor mağazası indirimi, 33 yaşındaki kadına fitness salonu reklamı. Kampanya 2'deki otomobil dergisi reklamı kimseye gösterilmez, çünkü 18-29 yaş arası A gelir grubu erkek kullanıcı yoktur (23 yaşındaki erkeğin gelir grubu C, A gelir grubundaki erkek ise 53 yaşındadır). Sonuç python ile de doğrulandı.

8. (derste sözlü, 02:28:28) Reklam örneğinde zincirdeki hiçbir reklam kullanıcıya uygun değilse ne olur? Bu nasıl önlenir?

> **Çözüm (derste verildi):** İstek `sonraki`'nin null olduğu halkaya kadar iletilir ve kullanıcıya hiç reklam gösterilmez. Önlemek için zincir doğru kurulmalı ve zincirin sonuna her kullanıcıya uygun bir reklam (örnekte `IndirimSporMagaza`) konmalıdır. Alternatif olarak bir halka iletmeden önce zincirin geri kalanının yanıt verip vermeyeceğini özyinelemeli sorgulayabilir, ama bu işi çok karmaşıklaştırır.

Dersin Toparlaması

Bu derste yapısal kalıplar tamamlandı ve davranışsal kalıplara geçildi. Facade, bir alt sistemin dağınık arayüzlerini tek ve üst düzey bir arayüzde toplayarak istekçilerle alt sistem arasındaki bağlaşımı azaltır; iç karmaşıklığı azaltmaz, ama alt sistemde yapılacak refactoring'in istekçilere yansımasını engeller. Flyweight, çok sayıda küçük nesneyi bir fabrika üzerinden paylaşarak bellek tasarrufu sağlar; nesnenin durumu paylaşılan içsel (intrinsic) ve işlem sırasında verilen dışsal (extrinsic) bilgi olarak ayrılır. Kalıbın koşullarının hepsi sağlanmalıdır ve kalıp nesne kimliğini korumaz. Kimya fabrikası örneğinde gram bilgisinin maddeden ayrılıp bileşene taşınması, bir verinin en yakın ilişkili olduğu sınıfta saklanması gerektiğini gösterdi; madde fabrikası da paylaşımı sağladı.

Proxy, asıl nesneyle aynı arayüzü gerçekleyen bir vekil koyarak erişimi denetler; resmi gerektiğinde yükleyen `ImageProxy` ve ödeme denetleyen `StreamProxy` (Protection Proxy) örnekleri işlendi ve Proxy'nin yalnız imza uyuşturan Adapter'dan farkı vurgulandı. Davranışsal kalıpların ilki olan Chain of Responsibility, bir isteği karşılayabilecek nesnelerden oluşan zincirde isteği en uygun halkaya kadar iletir; bağlama duyarlı yardım ve reklam kampanyası örneklerinde zincirin doğru kurulmasının, özellikle zincirin sonunda herkese hizmet verebilecek bir halka bulunmasının önemi görüldü. Reklam örneğinde ileride işlenecek Template Method kalıbının da kullanıldığı belirtildi. Hoca bir sonraki derse davranışsal kalıpların slaytlarından devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.